



République Tunisienne
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
École Nationale d'Ingénieurs de Sfax (ENIS)

Rapport Technique de Projet IoT

Système d'Irrigation Intelligente

Réalisé par :

Ghazza Ghorbel

Amina Abid

Molka Dammak

Encadré par :

Nouha Baccour

Classe : 2ème année / Génie Informatique

Année universitaire : 2025–2026

Table des matières

Introduction	3
1 Conception – Identification des besoins techniques	4
1.1 Problématique	4
1.2 Besoins fonctionnels et non fonctionnels	4
1.2.1 Besoins fonctionnels	4
1.2.2 Besoins non fonctionnels	5
1.3 Scénarios d'utilisation	5
1.4 Grandeurs physiques à mesurer	5
1.5 Actionneurs et contraintes associées	6
1.6 Exploitation des données (Data Analytics)	6
1.7 Contraintes environnementales	6
2 Définition de l'architecture globale du système	7
2.1 Identification des couches fonctionnelles	7
2.2 Identification des composants	7
2.3 Choix des technologies de communication	7
2.3.1 Dispositif → Gateway / Broker	7
2.3.2 Gateway / Edge → Cloud	8
2.3.3 Cloud → Application utilisateur	8
2.4 Schéma conceptuel de l'architecture	8
3 Conception de la partie dispositif	10
3.1 Architecture matérielle	10
3.1.1 Microcontrôleur (ESP8266)	10
3.1.2 Capteurs (YL-69, DHT22)	10
3.1.3 Actionneurs (LED / pompe)	10
3.1.4 Module de communication (WiFi)	10
3.1.5 Système d'alimentation	11
3.1.6 Interfaces matérielles	11
3.2 Architecture logicielle embarquée	11
3.2.1 Approche Bare-Metal ou RTOS	11
3.2.2 Stratégies d'économie d'énergie	11
3.2.3 Organisation des tâches	11
3.2.4 Bibliothèques utilisées	11

3.3	Flux de fonctionnement du dispositif	11
4	Conception de la logique applicative	12
4.1	Fonctionnalités principales de la plateforme IoT	12
4.1.1	Connectivité et ingestion des données	12
4.1.2	Device Management	12
4.1.3	Gestion des données	12
4.1.4	Traitement et règles	12
4.1.5	Analytics	12
4.1.6	API et support applicatif	13
4.1.7	Sécurité	13
4.2	Approche architecturale retenue (Edge + Cloud)	13
4.2.1	Partie Edge (local)	13
4.2.2	Partie Cloud	13
4.3	Flux de données de bout en bout	13
4.4	Schéma fonctionnel de la logique applicative	14

Introduction

1. Contexte et motivation

Face à la diminution des ressources hydriques et à l'augmentation des besoins agricoles en Tunisie, la gestion intelligente de l'irrigation devient une nécessité. Les méthodes traditionnelles d'irrigation entraînent souvent une surconsommation d'eau et d'énergie, ainsi qu'un manque d'adaptation aux besoins réels des cultures.

Dans ce contexte, l'utilisation des technologies IoT, combinée à des protocoles de communication légers comme MQTT et à des solutions Cloud telles que MongoDB Atlas, permet de proposer une solution efficace pour optimiser l'irrigation agricole.

2. Objectifs du projet

Ce projet vise à concevoir et implémenter un système d'irrigation intelligente basé sur l'Internet des Objets (IoT), capable de mesurer en temps réel les conditions environnementales, de prendre des décisions automatisées et de permettre une supervision à distance.

3. Portée et clientèle

Le système est destiné aux agriculteurs tunisiens souhaitant optimiser leur consommation d'eau et améliorer le rendement de leurs cultures grâce à l'automatisation et à l'analyse des données.

Chapitre 1

Conception – Identification des besoins techniques

1.1 Problématique

Face à la diminution des ressources hydriques et à l'augmentation des besoins agricoles en Tunisie, la gestion intelligente de l'irrigation devient une nécessité. Les méthodes traditionnelles d'irrigation entraînent souvent une surconsommation d'eau et d'énergie, ainsi qu'un manque d'adaptation aux besoins réels des cultures.

1.2 Besoins fonctionnels et non fonctionnels

1.2.1 Besoins fonctionnels

Le système d'irrigation intelligente doit assurer les fonctions suivantes :

- **Acquisition des données :**
 - Mesure de l'humidité du sol via le capteur YL-69.
 - Mesure de la température et de l'humidité de l'air via le capteur DHT22.
- **Traitement local :**
 - Utilisation du microcontrôleur ESP8266 pour traiter les données des capteurs.
- **Transmission des données :**
 - Envoi des données vers un broker MQTT (Mosquitto) via WiFi.
 - Utilisation du protocole MQTT pour une communication rapide et légère.
- **Traitement côté serveur :**
 - Collecte et traitement des données via Node-RED.
 - Stockage des données dans MongoDB Atlas.
- **Prise de décision :**
 - Activation/désactivation de l'irrigation (simulation via LED).
 - Automatisation basée sur des règles ou un modèle de Machine Learning.
- **Visualisation :**
 - Interface web développée en HTML, CSS et JavaScript.
 - Affichage des données en temps réel.
- **Alertes :**
 - Notification en cas de niveau critique d'humidité.

1.2.2 Besoins non fonctionnels

Le système doit respecter les contraintes suivantes :

- **Performance** : transmission rapide via MQTT ; traitement quasi temps réel avec Node-RED.
- **Fiabilité** : utilisation de Mosquitto pour une communication stable ; capteurs robustes adaptés à l’environnement agricole.
- **Sécurité** : communication sécurisée via TLS entre WildFly et MongoDB Atlas.
- **Scalabilité** : possibilité d’ajouter plusieurs capteurs ESP8266 ; architecture Cloud extensible.
- **Coût** : utilisation de composants low-cost (ESP8266, YL-69, DHT22).
- **Consommation énergétique** : ESP8266 optimisé pour faible consommation ; possibilité d’alimentation autonome.
- **Disponibilité** : accès aux données à distance via serveur WildFly.

1.3 Scénarios d’utilisation

Scénario 1 : Irrigation automatique

Le capteur YL-69 mesure l’humidité du sol. L’ESP8266 envoie les données via MQTT. Node-RED traite les données. Si l’humidité est inférieure au seuil, la LED est activée (simulation pompe).

Scénario 2 : Surveillance à distance

L’utilisateur consulte les données via une interface web. Les données sont stockées dans MongoDB Atlas.

Scénario 3 : Alerte

Si les conditions sont critiques, une alerte est générée via le système.

Scénario 4 : Optimisation intelligente

Les données stockées sont analysées. Un modèle ML ajuste automatiquement les cycles d’irrigation.

1.4 Grandeurs physiques à mesurer

TABLE 1.1 – Grandeurs physiques mesurées

Grandeur	Capteur	Type de sortie	Fréquence
Humidité du sol	YL-69	Analogique	5–10 minutes
Température de l’air	DHT22	Numérique	2 secondes
Humidité de l’air	DHT22	Numérique	2 secondes

Les données sont de petite taille, transmises périodiquement via MQTT et stockées sous forme de séries temporelles dans MongoDB Atlas.

1.5 Actionneurs et contraintes associées

Le système inclut comme actionneur principal une **LED** (simulation de pompe) :

- **Rôle** : simuler l'activation de l'irrigation.
- **Commande** : via GPIO de l'ESP8266.
- **Contraintes** : réponse immédiate, faible consommation.

Dans un système réel, la LED sera remplacée par une pompe ou une électrovanne.

1.6 Exploitation des données (Data Analytics)

Les données seront exploitées à plusieurs niveaux :

- **Visualisation** : interface web (HTML/CSS/JS).
- **Traitement en temps réel** : Node-RED pour automatisation.
- **Stockage** : MongoDB Atlas (Cloud).
- **Communication** : MQTT via Mosquitto.
- **Analyse avancée** : modèle de Machine Learning, prédiction des besoins en irrigation, optimisation des cycles.

1.7 Contraintes environnementales

- **Environnement** : extérieur (agriculture), température élevée et humidité variable.
- **Contraintes matérielles** : capteurs exposés à la poussière et à l'eau.
- **Contraintes réseau** : dépendance au WiFi (ESP8266) ; possibilité de latence ou perte de connexion.
- **Maintenance** : accès limité, besoin d'un système autonome et robuste.

Chapitre 2

Définition de l'architecture globale du système

L'architecture globale de notre système d'irrigation intelligente est conçue pour assurer une collecte, un traitement et une visualisation efficaces des données tout en garantissant fiabilité, scalabilité et autonomie énergétique.

2.1 Identification des couches fonctionnelles

Le système IoT peut être décomposé en cinq couches principales :

1. Couche perception (capteurs et actionneurs)

- YL-69 : mesure de l'humidité du sol.
- DHT22 : mesure de la température et de l'humidité de l'air.
- LED, remplacée à terme par une électrovanne ou pompe réelle.

2. Couche réseau

- Transmission des données via WiFi depuis l'ESP8266.
- Protocole MQTT (Mosquitto Broker).
- Flux : ESP8266 vers le Broker MQTT ; Serveur vers l'ESP8266.

3. Couche edge (optionnelle)

- L'ESP8266 agit comme edge device pour le traitement local et les décisions simples.

4. Couche plateforme IoT / middleware

- Node-RED : orchestration des flux de données et automatisation.
- MongoDB Atlas : stockage et historisation dans le Cloud.
- WildFly : serveur applicatif Java pour les communications sécurisées.

5. Couche application

- Interface utilisateur web (HTML, CSS, JavaScript).
- Visualisation en temps réel, alertes et contrôle manuel des actionneurs.

2.2 Identification des composants

2.3 Choix des technologies de communication

2.3.1 Dispositif → Gateway / Broker

- **Protocole** : MQTT (Message Queuing Telemetry Transport).

TABLE 2.1 – Composants du système par couche

Couche	Composant	Rôle
Perception	ESP8266	Collecte des données capteurs, contrôle actionneurs
	YL-69	Mesure l'humidité du sol
	DHT22	Mesure température et humidité de l'air
	LED	Simule la pompe
Réseau	WiFi	Connectivité sans fil entre ESP8266 et Broker MQTT
	Mosquitto	Gestion de la communication publish/subscribe MQTT
Edge	ESP8266	Traitement local et exécution de commandes simples
Middleware	Node-RED	Orchestration et traitement des flux de données
	MongoDB Atlas	Stockage des mesures et historique
	WildFly	Serveur applicatif sécurisé, communication et API
Application	Web App	Visualisation et contrôle utilisateur

- **Connectivité** : WiFi intégré à l'ESP8266.
- **Avantages** : léger et rapide ; communication bidirectionnelle (pub/sub).

2.3.2 Gateway / Edge → Cloud

- **Protocole** : HTTPS ou MQTT sur TLS.
- **Technologie** : Node-RED vers MongoDB Atlas.
- **Avantages** : sécurité renforcée (TLS) ; scalabilité pour plusieurs dispositifs.

2.3.3 Cloud → Application utilisateur

- **Protocole** : HTTP/HTTPS via API REST.
- **Technologie** : Interface web HTML/CSS/JS.
- **Avantages** : accès à distance en temps réel ; visualisation graphique ; contrôle manuel.

2.4 Schéma conceptuel de l'architecture

L'architecture suit un modèle en cinq couches, de la perception physique jusqu'à l'application utilisateur.

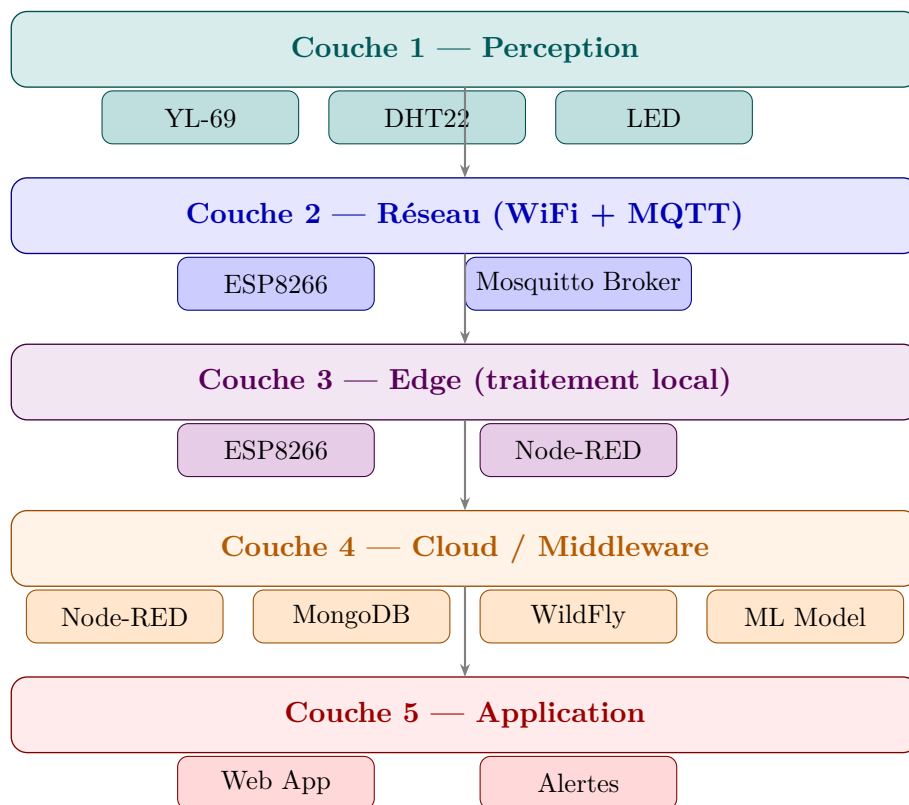


FIGURE 2.1 – Architecture globale du système IoT en cinq couches

Chapitre 3

Conception de la partie dispositif

3.1 Architecture matérielle

L'architecture matérielle définit les composants physiques du nœud IoT (capteurs, actionneurs, microcontrôleur) et leur interconnexion.

3.1.1 Microcontrôleur (ESP8266)

- Microcontrôleur économique et faible consommation.
- Intègre le module WiFi pour la communication MQTT.
- Nombreux GPIO pour connecter capteurs et actionneurs.
- Possibilité de traitement local pour automatisation simple (edge computing).

3.1.2 Capteurs (YL-69, DHT22)

YL-69 – Humidité du sol :

- Mesure l'humidité par conductivité électrique du sol.
- Sortie analogique, fréquence de mesure configurable.

DHT22 – Température et humidité de l'air :

- Sortie numérique, facile à interfacer avec l'ESP8266.
- Données mises à jour toutes les 2 secondes.

3.1.3 Actionneurs (LED / pompe)

- LED connectée via une résistance pour correspondre à la tension du GPIO.
- Commandée par l'ESP8266 pour simuler l'irrigation.

Dans une version finale, la LED sera remplacée par une pompe ou électrovanne pour irriguer réellement les cultures.

3.1.4 Module de communication (WiFi)

- WiFi intégré à l'ESP8266.
- Protocole MQTT via Mosquitto Broker.
- Communication bidirectionnelle pour capteurs et actionneurs.

3.1.5 Système d'alimentation

- Alimentation 5V ou 3,3V selon la configuration de l'ESP8266.
- Possibilité d'alimentation solaire pour rendre le dispositif autonome.

3.1.6 Interfaces matérielles

- Connexion capteurs : GPIO et broches analogiques de l'ESP8266.
- Connexion actionneurs : GPIO.
- Interface de programmation : USB ou UART pour le développement et le debug.

3.2 Architecture logicielle embarquée

3.2.1 Approche Bare-Metal ou RTOS

- **Bare-metal** : code exécuté directement sur l'ESP8266 sans OS complet ; contrôle total des capteurs et actionneurs ; suffisant pour un dispositif simple.
- Un RTOS peut être envisagé si plusieurs tâches complexes doivent être gérées simultanément.

3.2.2 Stratégies d'économie d'énergie

- Mode veille (*deep sleep*) de l'ESP8266 entre deux mesures.
- Activation des capteurs uniquement lors des acquisitions.
- Gestion de la LED/pompe uniquement lorsque nécessaire.

3.2.3 Organisation des tâches

1. **Acquisition des mesures** : lecture du YL-69 (humidité du sol) et du DHT22 (température et humidité de l'air).
2. **Transmission des données** : publication via MQTT sur Mosquitto Broker.
3. **Traitement local** : comparaison avec le seuil ; activation/désactivation de la LED/pompe selon les résultats.
4. **Surveillance et debug** : sortie des valeurs sur le port série.

3.2.4 Bibliothèques utilisées

- `DHT.h` (Adafruit) : communication avec le capteur DHT22.
- `PubSubClient.h` (Nick O'Leary) : communication MQTT.

3.3 Flux de fonctionnement du dispositif

Le dispositif suit un cycle continu : **Acquisition** → **Traitement local** → **Transmission MQTT** → **Commande actionneur** → **Mode veille**.

Chapitre 4

Conception de la logique applicative

La logique applicative correspond à l'organisation et au traitement des données collectées par le système IoT. Elle définit comment les données sont ingérées, stockées, analysées et exposées aux utilisateurs, tout en garantissant sécurité, fiabilité et réactivité.

4.1 Fonctionnalités principales de la plateforme IoT

4.1.1 Connectivité et ingestion des données

- Broker MQTT (Mosquitto) pour la communication entre capteurs/actionneurs et serveur.
- Transmission bidirectionnelle :
 - Capteurs vers Broker : publication des mesures (humidité, température).
 - Serveur vers Capteurs : commandes d'actionneurs (LED/pompe).

4.1.2 Device Management

- Enregistrement des dispositifs (ESP8266).
- Surveillance de l'état des capteurs et actionneurs.
- Mise à jour du firmware via OTA (*Over-The-Air*).

4.1.3 Gestion des données

- **Stockage temps réel et historique** : MongoDB Atlas ; données en séries temporelles pour faciliter l'analyse.
- Historisation pour analyses futures et génération de rapports.

4.1.4 Traitement et règles

- Node-RED comme moteur de règles : filtrage, transformation, déclenchement automatique.
- Seuils d'humidité du sol et alertes en cas de conditions critiques.

4.1.5 Analytics

- Détection d'anomalies dans les mesures (capteur défectueux, valeurs aberrantes).
- Modèles prédictifs pour ajuster les cycles d'irrigation selon les besoins des plantes.

4.1.6 API et support applicatif

- WildFly : serveur applicatif exposant les données via API REST.
- Permet à l'interface web HTML/CSS/JS de récupérer les données en temps réel.

4.1.7 Sécurité

- Chiffrement TLS pour toutes les communications (serveur, MongoDB, capteurs).
- Gestion des identités et contrôle d'accès pour l'application web.

4.2 Approche architecturale retenue (Edge + Cloud)

Pour ce projet, nous adoptons une **architecture hybride (Edge + Cloud)** :

4.2.1 Partie Edge (local)

- ESP8266 : acquisition et traitement local des données.
- Broker MQTT local (Mosquitto) : transmission rapide et fiable.
- Réaction rapide aux événements sans dépendance à Internet.
- Filtrage et agrégation simples avant envoi au Cloud.

4.2.2 Partie Cloud

- Node-RED : orchestration, règles, transformation et filtrage avancé.
- MongoDB Atlas : stockage historique et séries temporelles.
- WildFly : API REST pour l'interface utilisateur.
- Analytics avancée : détection d'anomalies, prédiction des besoins en irrigation.

Cette approche hybride garantit la **continuité de service** même en cas de perte de connexion Internet, tout en offrant des fonctionnalités avancées via le Cloud.

4.3 Flux de données de bout en bout

1. **Acquisition** : Capteurs (YL-69, DHT22) vers l'ESP8266.
2. **Traitement local** : vérification des seuils ; activation de l'actionneur si nécessaire.
3. **Transmission Edge vers Cloud** : ESP8266 vers Mosquitto Broker vers Node-RED.
4. **Stockage Cloud** : Node-RED vers MongoDB Atlas.
5. **Analyse et règles** : Node-RED applique les règles, déclenche les alertes et agrège les données.
6. **Exposition des données** : WildFly vers API REST vers Interface web.
7. **Interaction utilisateur** : Application web vers API REST vers Contrôle actionneurs.

4.4 Schéma fonctionnel de la logique applicative

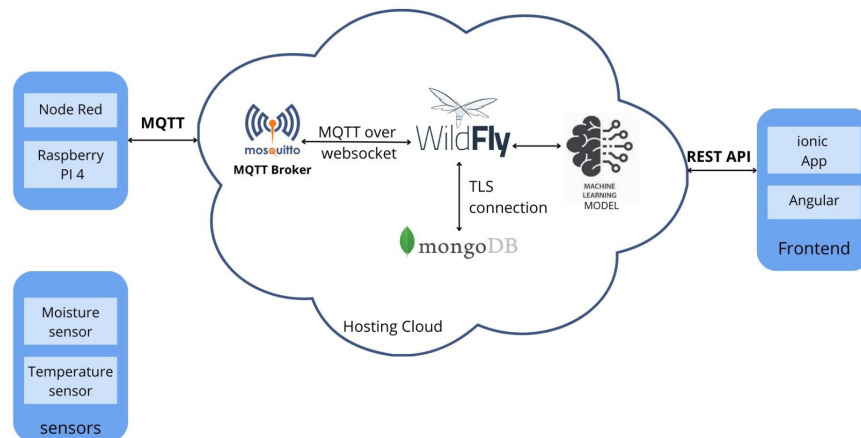


FIGURE 4.1 – Schéma fonctionnel de l'architecture Edge + Cloud

Description synthétique :

- Le traitement immédiat se fait sur l'ESP8266 (réaction rapide aux seuils).
- Les données sont transmises au Cloud pour stockage historique et analyses avancées.
- L'utilisateur peut consulter les mesures, recevoir des alertes et contrôler le système via l'application web.